LÖSUNGEN · LEHREREXEMPLAR

W4·1 – Stapel und Schlange: Lösungen

Zum Arbeitsblatt `AQ_W4_AB1_Stapel_und_Schlange.docx` · IDE: `AQ_W4_L4_Hafen_LSG.txt` (Hafen, Zugbuch, Initiative) · alle Zustände, Rückgaben und Laufzeitfehler mit Java (javac 21) und exakten Nachbauten der Klassen Stack und Queue (Object-basiert, gleiche Exceptions) nachgerechnet; Zugbuch und Initiative sind die Musterlösung von Level 4.2 · Schreibweise: Stapel oben zuerst, Schlange vorne zuerst, – = leer

Teil 1 (DS 24)

Aufgabe 1 – Teller und Kasse (5 BE)

a) Die vier oberen Teller nacheinander abnehmen und auf die freie Ablage legen (dort liegen sie in umgekehrter Reihenfolge), den untersten nehmen, dann die vier Teller einzeln zurücklegen – zweimal umgedreht ergibt wieder die alte Reihenfolge. **9 Bewegungen** (4 + 1 + 4). Das ist schon das Muster „Hilfsstapel“ aus DS 26 (2 BE).

b) Zuerst bezahlt **A**, nach Ds Ankunft bezahlt **B**; vorne steht jetzt **C**, dahinter D. Eine „Stapel-Kasse“ bediente den zuletzt Gekommenen zuerst – wer früh kam, wartet, solange immer neue Kunden kommen, im schlimmsten Fall ewig (2 BE). **c)** z. B. LIFO: Aufrufstapel bei Rekursion (Welt 3), Rückgängig im Editor, Zurück-Knopf im Browser; FIFO: Druckerwarteschlange, Tastaturpuffer, Anfragen an einen Server (1 BE).

Aufgabe 2 – Kistenturm und Schiffsschlange (8 BE)

Zeile	s (oben → unten)	Rückgabe
2	–	true
3	42	–
4	137, 42	–
5	42	137
6	42, 42	– (top() liefert 42)
7	18, 42, 42	–
8	18, 42, 42	18
9	18, 42, 42	false

Zeile	q (vorne → hinten)	Rückgabe
2	–	true
3	42	–
4	42, 137	–
5	42, 137, 81	–
6	42, 137, 81	42
7	137, 81, 42	– (dequeue() liefert 42)
8	81, 42	137

a) und **b)** je 3 BE. Typische Fehler: Turm von oben nach unten gefüllt; nach `top()` bzw. `head()` ein Element entfernt; in Zeile 7 von b) das Element vorne statt hinten wieder eingereiht. Zeile 6 in a) enthält zwei Aufrufe: `top()` liefert 42, `push` hat keine Rückgabe.

c) Ohne Zeile 3 liegt nach Zeile 4 nur 137 auf dem Stapel; Zeile 5 entnimmt sie, der Stapel ist leer. In **Zeile 6** führt `s.top()` zum Laufzeitfehler (Java: IllegalStateException) (1 BE). **d)** `if (!s.isEmpty()) { s.push(s.top()); }` (1 BE).

Aufgabe 3 – Kräne im Wechselspiel (8 BE)

Nr.	Anweisung	q (vorne → hinten)	s (oben → unten)	Bemerkung
0	`(Start)`	5, 1, 8	7, 2, 9	Startzustand
1	`s.push(q.dequeue())`	1, 8	5, 7, 2, 9	5 wandert von q nach s
2	`s.push(s.pop())`	1, 8	5, 7, 2, 9	entnehmen und sofort wieder auflegen
3	`q.enqueue(q.dequeue())`	8, 1	5, 7, 2, 9	rotieren: 1 wandert nach hinten

Nr.	Anweisung	q (vorne → hinten)	s (oben → unten)	Bemerkung
4	`q.enqueue(s.pop())`	8, 1, 5	7, 2, 9	5 wandert zurück ans Ende von q
5	`int x = (Integer) s.pop() + (Integer) q.head()`	8, 1, 5	2, 9	pop entnimmt 7, head liest 8: x = 15
6	`s.push(x)`	8, 1, 5	15, 2, 9	x wird gepusht
7	`q.enqueue(s.top())`	8, 1, 5, 15	15, 2, 9	top liest nur: 15 bleibt auf s

a) 4 BE (je zwei Zeilen 1 BE). Endzustand q = 8, 1, 5, 15 und s = 15, 2, 9; x = 15.

b) Zeile 2 entnimmt das oberste Element und legt es sofort wieder **oben** auf – es landet auf demselben Platz, s bleibt gleich. Zeile 3 entnimmt das **vorderste** Element, stellt es aber **hinten** an: Es wandert ans Ende, die Reihenfolge ändert sich (Rotieren um eine Position) (2 BE).

c) `pop()` und `head()` liefern den Typ Object; mit Object kann man nicht rechnen – erst der Cast `(Integer)` macht daraus eine Zahl (Unboxing). Beim Hineinlegen erwarten `push`/`enqueue` ein Object: Die Zahl bzw. x wird automatisch verpackt, und `s.top()` liefert bereits ein Object – kein Cast nötig. Ist s vor Zeile 5 leer, führt `s.pop()` zum Laufzeitfehler (IllegalStateException); Absicherung z. B. `if (! s.isEmpty() && !q.isEmpty()) { … }` (2 BE).

Aufgabe 4 – Imbiss, Containerschiff, Rückgängig, Drucker (10 BE)

Anwendung	Struktur	Begründung (Kernpunkte)
(1) Schnellimbiss	Schlange	FIFO: Wer zuerst bestellt, wird zuerst bedient. Eingabe = enqueue (hinten), Küche = dequeue (vorne).
(2) Containerschiff	Stapel	LIFO: Nur der oberste Container ist greifbar – push beim Laden, pop/top beim Entladen.
(3) Texteditor	Stapel	Die zuletzt gemachte Änderung wird zuerst zurückgenommen (LIFO); jede Änderung = push, Strg+Z = pop.
(4) Drucker	Schlange	FIFO in der Reihenfolge des Eingangs: neue Aufträge hinten, gedruckt wird vorne.
(5) Drucker im Lehrerzimmer	dynamische Reihung	Vorziehen = Einfügen an beliebiger Position (insertAt), Abbrechen = Löschen aus der Mitte (delete(i)) – Stapel und Schlange erlauben Zugriff nur oben bzw. vorne.

a) je 1 BE (Struktur und Begründung mit Zugriffsprinzip). **b)** Stapel: **10, 18, 5, 24, 3** (oben zuerst, 3 liegt unten); Schlange: **3, 24, 5, 18, 10** (vorne zuerst) (2 BE).

c) Stapel: `pop()` (10), `x = (Integer) pop()` (18 merken), `pop()` (5), `push(x)`, `push(6)` → **5 Aufrufe**, s = 6, 18, 24, 3 (oben zuerst) – von unten gelesen 3, 24, 18, 6. Schlange: `enqueue(dequeue())` für 3 und für 24, `dequeue()` (5 weg), `enqueue(dequeue())` für 18, `dequeue()` (10 weg), `enqueue(6)` → **9 Aufrufe**, q = 3, 24, 18, 6. Beim Stapel liegen die zu ändernden Werte oben und sind schnell erreichbar; bei der Schlange muss sie einmal komplett „durchrotiert“ werden. Für Änderungen in der Mitte ist eine dynamische Reihung besser (3 BE).

Aufgabe 5 – Level 4.1

Musterlösungen und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klasse Hafen (10 Tests). Häufig: obersterWert mit pop statt top, leeren mit if statt while, ", " nach jedem Namen, umkehren mit Hilfsschlange (nichts wird umgekehrt). Kontrollwerte aus dem Hauptprogramm: Kistenturm (von oben) 16, 15, 8, 4 · oberste Nummer 16 · Warteschlange Albatros, Moewe, Kormoran.

Teil 2 (DS 25, Kür)

Aufgabe 6 – Das Zugbuch (9 BE)

Nr.	Aufruf	Rückgabe	erledigt	zurueckgenommen
1	`zug("Nord")`	–	Nord	–
2	`zug("Ost")`	–	Ost, Nord	–
3	`rueckgaengig()`	"Ost"	Nord	Ost
4	`zug("Sued")`	–	Sued, Nord	–
5	`zug("West")`	–	West, Sued, Nord	–
6	`rueckgaengig()`	"West"	Sued, Nord	West
7	`rueckgaengig()`	"Sued"	Nord	Sued, West
8	`wiederholen()`	"Sued"	Sued, Nord	West
9	`anzahlZuege()`	2	Sued, Nord	West
10	`rueckgaengig()`	"Sued"	Nord	Sued, West
11	`rueckgaengig()`	"Nord"	–	Nord, Sued, West
12	`rueckgaengig()`	""	–	Nord, Sued, West

a) 5 BE. Zeile 4: Der neue Zug „Sued“ leert zurueckgenommen – „Ost“ ist endgültig vergessen. Zeile 9 lässt beide Stapel unverändert. Zeile 12: erledigt ist leer → "" (kein Laufzeitfehler dank Leer-Abfrage).

b) Ohne Leeren läge in zurueckgenommen nach Zeile 8 **West, Ost** (statt nur West). Der nächste Aufruf wiederholen() liefert West (richtig), der übernächste aber **„Ost“** – ein Zug, der vor „Sued“ und „West“ zurückgenommen wurde und zu einer verworfenen Spielgeschichte gehört. erledigt wäre dann Ost, West, Sued, Nord – so wurde nie gespielt (2 BE).

c) Beim vierten Zug müsste der **älteste** Zug vergessen werden – er liegt ganz **unten** im Stapel und ist nur nach vollständigem Umschichten (Hilfsstapel) erreichbar: aufwendig und fehleranfällig. Besser: eine dynamische Reihung (neuer Zug: append, ältester: delete(0), rückgängig: letztes Element über getLength() – 1) (2 BE).

Aufgabe 7 – Initiative im Kampf (8 BE)

Nr.	Aufruf	Rückgabe	reihe (vorne → hinten)
1	`beitreten("Aria")`	–	Aria
2	`beitreten("Borin")`	–	Aria, Borin
3	`beitreten("Cyra")`	–	Aria, Borin, Cyra
4	`beitreten("Goblin")`	–	Aria, Borin, Cyra, Goblin
5	`amZug()`	"Aria"	Borin, Cyra, Goblin, Aria
6	`amZug()`	"Borin"	Cyra, Goblin, Aria, Borin
7	`amZug()`	"Cyra"	Goblin, Aria, Borin, Cyra
8	`ausscheiden("Aria")`	true	Goblin, Borin, Cyra
9	`amZug()`	"Goblin"	Borin, Cyra, Goblin
10	`amZug()`	"Borin"	Cyra, Goblin, Borin
11	`ausscheiden("Ork")`	false	Cyra, Goblin, Borin
12	`amZug()`	"Cyra"	Goblin, Borin, Cyra

a) 4 BE. Zeile 8: Aria wird aus der Mitte entfernt, Goblin bleibt vorne. Zeile 11: „Ork“ ist nicht dabei → falsch, reihe unverändert.

b) Vorher Goblin, Aria, Borin, Cyra. Die fehlerhafte Fassung rotiert, bis Aria vorne steht (Goblin wandert nach hinten), und entnimmt sie: **Borin, Cyra, Goblin** – als Nächster wäre **Borin** am Zug. Richtig (Zeile 8): Goblin, Borin, Cyra, als Nächster **Goblin**. Goblin verliert seinen Zug, die Reihenfolge ist verschoben (2 BE).

c) (1) name ist **nicht** in der Reihe (z. B. „Ork“): Die Schleife rotiert endlos – das Programm hängt (Endlosschleife), statt falsch zu liefern. (2) Die Reihe ist **leer**: `reihe.head()` führt sofort zum Laufzeitfehler (IllegalStateException). Außerdem liefert die Methode nie falsch (2 BE).

Aufgabe 8 – Level 4.2

Musterlösung und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klassen Zugbuch und Initiative (10 Tests). Kontrollwerte aus dem Hauptprogramm: Züge 3 · Rückgängig Sued, Ost · Wiederholen Ost · Züge 2 · Am Zug Aria Borin Goblin Aria · nach ausscheiden("Goblin"): Borin Aria Borin Aria.

Aufgabe 9 – Bug-Hunt im Hafenbüro (8 BE)

a) Zeile 2: `s.pop()` liefert Object; ein Object kann einer Variablen vom Typ Kiste nicht ohne Cast zugewiesen werden (javac: „incompatible types: Object cannot be converted to Kiste“). Richtig: `Kiste k1 = (Kiste) s.pop();` (1 BE).

b) Eine Kiste: Zeile 2 entnimmt sie, in **Zeile 3** führt `top()` auf dem nun leeren Stapel zum Laufzeitfehler. Leerer Stapel: schon **Zeile 2** (`pop()`) – jeweils IllegalStateException (mit Java geprüft) (2 BE).

```
public static int obersteZwei(Stack s) {
    if (s.isEmpty()) {               // Leer?
        return 0;
    }
    Kiste k1 = (Kiste) s.pop();
    int summe = k1.getGewicht();
    if (!s.isEmpty()) {              // Leer? – vor top
        Kiste k2 = (Kiste) s.top();  // nur lesen
        summe = summe + k2.getGewicht();
    }
    s.push(k1);                      // dieselbe Kiste zurück
    return summe;
}
```

c) 3 BE (Leer-Abfrage vor pop, Leer-Abfrage vor top, k1 zurücklegen bzw. Stapel unverändert). Mit Java geprüft: Standard-Ladung → **14** (Wein 6 + Tuch 8), s danach Wein, Tuch, Salz, Gold, Erz, Holz; eine Kiste Salz → 3; leer → 0. Gleichwertig: zweite Kiste ebenfalls mit pop holen und beide in richtiger Reihenfolge zurücklegen (erst k2, dann k1).

d) Ein Cast von Object nach Integer ist für den Compiler erlaubt – das Objekt **könnte** ja eine Zahl sein; geprüft wird erst beim Ausführen. Dort ist es eine Kiste → Laufzeitfehler **ClassCastException** (2 BE).

Für Schnelle

Palindrom: Der Stapel liefert die Zeichen von hinten, die Schlange von vorne – stimmen alle Paare überein, liest sich das Wort vorwärts wie rückwärts. Mit Java geprüft: RENTNER → wahr, HAFEN → falsch, "" → wahr (true false true). **Kranfolgen:** Bei drei Kisten sind genau **5** Reihenfolgen möglich: 1 2 3 · 1 3 2 · 2 1 3 · 2 3 1 · 3 2 1. **3, 1, 2 ist unmöglich:** Wird 3 zuerst abgegeben, liegen 1 und 2 noch auf dem Stapel, 2 über 1 – also kommt 2 vor 1. Bei vier Kisten sind es **14** (Catalan-Zahlen 1, 2, 5, 14, 42 …).